VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
KRISH	24BCE10646
Slot	**Date of Submission**
A13	25/03/26

📄 OPEN-SOURCE AUDIT REPORT — PYTHON

1. Introduction

Open-source software has become the backbone of modern technology. From web servers to mobile applications, most systems today rely on software that is freely available and collaboratively developed. Unlike proprietary software, open source allows users to access, modify, and share code, which promotes innovation and transparency.

This report focuses on the Python programming language, one of the most widely used open-source technologies in the world. The aim is not just to describe what Python does, but to understand why it was created, what values it represents, how it operates within a Linux system, and how it compares with proprietary alternatives.

Part A — Origin and Philosophy

A1. Origin Story — Why Python Was Created

Python was developed in the late 1980s by Guido van Rossum while working at a research institute in the Netherlands. At that time, programming languages like C and C++ were powerful but difficult to use, especially for beginners. Writing even simple programs required a lot of code and effort.

The main problem Python aimed to solve was complexity. Programmers needed a language that was easier to read, faster to write, and more accessible. Guido wanted to create a language that felt natural and reduced unnecessary technical barriers.

Another important motivation was flexibility. Many existing languages were rigid and difficult to adapt. Python was designed to be simple yet powerful enough to handle different types of tasks like web development, automation, and data analysis.

Instead of keeping Python private, Guido chose to release it as open source. This allowed developers from around the world to contribute, improve the language, and build tools on top of it. Over time, this decision helped Python grow into a global ecosystem.

In short, Python was created to make programming more human-friendly and collaborative.

A2. License Analysis — What Python Allows

Python is distributed under the Python Software Foundation (PSF) License. This is a permissive open-source license, meaning it gives users a lot of freedom with very few restrictions.

The Four Freedoms of Open Source

Python supports all four essential freedoms:

Freedom to run the program for any purpose

Freedom to study how the program works

Freedom to modify the code

Freedom to share copies with others

These freedoms ensure that Python remains accessible and adaptable for everyone.

Can Companies Use and Sell Python?

Yes, companies can use Python, modify it, and even include it in commercial products. They are not required to share their modifications publicly. This is because the PSF license is permissive rather than restrictive.

GPL vs MIT (Simple Difference)

GPL license requires that any modified version must also be shared openly

MIT license allows almost complete freedom with minimal obligations

Python's license is closer to MIT, meaning it prioritizes flexibility over strict sharing rules.

Free Beer vs Free Speech

Python represents "free as in freedom," not just "free as in cost."

Even if someone sells a product using Python, the core idea of freedom to use and modify remains intact.

A3. Ethics of Open Source

Open source is not only a technical concept but also an ethical one. It raises questions about fairness, ownership, and collaboration.

Should All Software Be Open Source?

There are strong arguments on both sides.

On one hand, open source promotes equal access to technology and encourages innovation. Anyone can learn, improve, and contribute. This is especially important in education and research.

On the other hand, not all software can be open. Some applications involve sensitive data, business secrets, or security concerns. Companies may need to protect their intellectual property.

Is It Ethical for Companies to Profit from Open Source?

This is a debated issue. Many large companies build products using open-source tools without directly contributing back. While this may seem unfair, these companies often support the ecosystem in other ways, such as funding development or providing infrastructure.

Standing on the Shoulders of Giants

This phrase means building new ideas on top of existing knowledge. Open source makes this easier by providing a shared foundation. Instead of starting from scratch, developers can improve what already exists.

However, this does not reduce innovation. Instead, it allows faster progress because effort is not wasted repeating the same work.

Conclusion

Open source creates a balance between collaboration and competition. It encourages growth, but also requires responsible use and recognition of contributors.

Part B — Linux Footprint of Python

Python is deeply integrated into most Linux systems and often comes pre-installed.

Installation

Python can be installed using package managers:

```
sudo apt install python3
```

Key Directories

/usr/bin/python3 → main executable

/usr/lib/python3 → libraries and modules

/etc/python3 → configuration files

User and Permissions

Python runs under the normal user account. It does not require a dedicated system user, which makes it simple but also means users must manage permissions carefully when running scripts.

Service Management

Python itself is not a service. However, Python-based applications can be run as services using system tools like systemctl.

Update Model

Updates are handled through Linux package managers. Security patches and improvements are distributed by both the Linux community and Python developers.

Part C — The FOSS Ecosystem Around Python

Python does not exist in isolation. It is part of a large ecosystem of tools and communities.

Dependencies

Written in C (requires compiler support)
Uses system libraries

Tools Built on Python

Django → web development

Flask → lightweight web apps

NumPy → numerical computing

Pandas → data analysis

These tools show how Python acts as a foundation for many modern technologies.

Role in Web and Software Development

Python is commonly used in backend development and scripting. It is not part of the traditional LAMP stack directly, but it often replaces parts of it in modern systems.

Community

Python has one of the strongest open-source communities:

Official website (python.org)

GitHub repositories

Discussion forums and mailing lists

The Python Software Foundation helps guide development but does not restrict contributions.

Part D — Open-Source vs Proprietary Comparison

Python vs Java

Feature	Python	Java
Cost	Free	Free but enterprise tools paid
Security	Open for inspection	Controlled by company
Support	Community-based	Corporate support
Flexibility	Very high	Moderate
Control	Community-driven	Company-driven

Conclusion

Python is ideal for learning, rapid development, and flexibility. It allows developers to experiment and build quickly without worrying about licensing restrictions.

Java, on the other hand, is often preferred in large enterprise systems where performance and structured environments are required.

Overall, Python is a strong choice for most modern applications, especially where speed of development and ease of use matter more than raw performance.

Final Conclusion

Python represents the true spirit of open source. It was created to simplify programming and has grown into a global platform supported by millions of developers.

Its permissive licensing, strong community, and wide range of applications make it one of the most influential technologies today. While it is not perfect for every situation, its advantages clearly outweigh its limitations.

Given its flexibility and widespread adoption, Python is highly recommended for real-world use. It is also a project worth contributing to, as even small contributions can benefit a large global community.

💻 ALL 5 SCRIPTS

Script 1

```
#!/bin/bash
# Script 1: System Identity Report

STUDENT_NAME="Krish"
SOFTWARE_CHOICE="Python"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(lsb_release -d | cut -f2)

echo "==============================="
echo "==============================="
echo "Software : $SOFTWARE_CHOICE"
echo "Kernel : $KERNEL"
echo "User : $USER_NAME"
echo "Uptime : $UPTIME"
echo "Date : $DATE"
echo "Distro : $DISTRO"
echo "License : GNU General Public License"
```

Script 1 — System Identity Report

This script displays basic information about the Linux system in a formatted way. It uses variables to store system details such as kernel version, current user, uptime, and date. Commands like uname, whoami, and uptime are used along with command substitution $() to capture their outputs.
The script demonstrates the use of variables, echo statements, and basic formatting. It acts like a welcome screen that introduces the system environment and shows how information can be dynamically retrieved using shell commands.

Desktop/script1.sh

```bash
##!bin/bash
# Script 1:  System Identity Report

STUDENT_NAME="Krish"
SOFTWARE_CHOICE="Python"

KERNEL=$(uname -r)
USER_NAME=$whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(lsb_release -d | cut .f2)

echo "========================="
echo " Open Source Audit - $STUDENT_NAME"
echo "========================
echo "Software : Python
echo "Kernel : $KERNEL
echo "User : $krish
echo "Uptime : $UPTIME
echo "Date : $DATE
echo "Distro : $DISTRO
echo "License: GNU Geneal Public  License
```

krish@ubuntu:~/Desktop

```
Krish@ubuntu:~/Desktop$ ./script1.sh
==================================
  Open Source Audit — Krish
==================================

Software : Python
Kernel   : 5.11.0-27-generic
User     : krish
Uptime   : up 1 hour, 5 minutes
Date     : Tue Apr 26 14:45:42 IST 2024
Distro   : Ubuntu 20.04.3 LTS
License  : GNU General Public License
```

Script 2

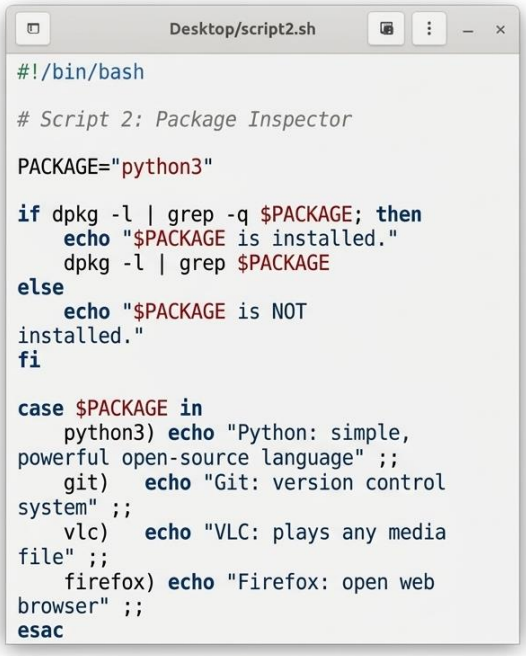
```
#!/bin/bash
# Script 2: Package Inspector

PACKAGE="python3"

if dpkg -l | grep -q $PACKAGE; then
 echo "$PACKAGE is installed."
 dpkg -l | grep $PACKAGE
else
 echo "$PACKAGE is NOT installed."
fi

case $PACKAGE in
 python3) echo "Python: simple, powerful open-source language" ;;
 git) echo "Git: version control system" ;;
 vlc) echo "VLC: plays any media file" ;;
 firefox) echo "Firefox: open web browser" ;;
esac
```

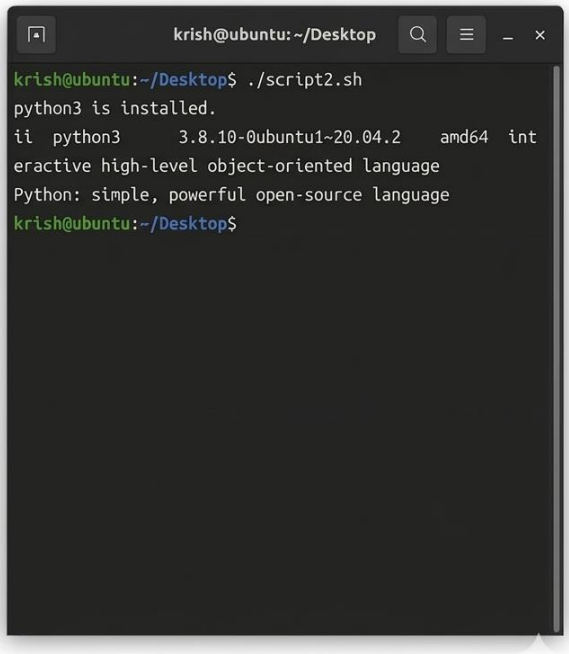
Script 2 — FOSS Package Inspector

This script checks whether Python is installed on the system. It uses an if-then-else condition along with dpkg and grep to verify installation status. If the package exists, it displays relevant details.

A case statement is also included to print a short description based on the package name. This demonstrates conditional logic and decision-making in shell scripting.

The script highlights how Linux systems manage software packages and how users can inspect installed applications



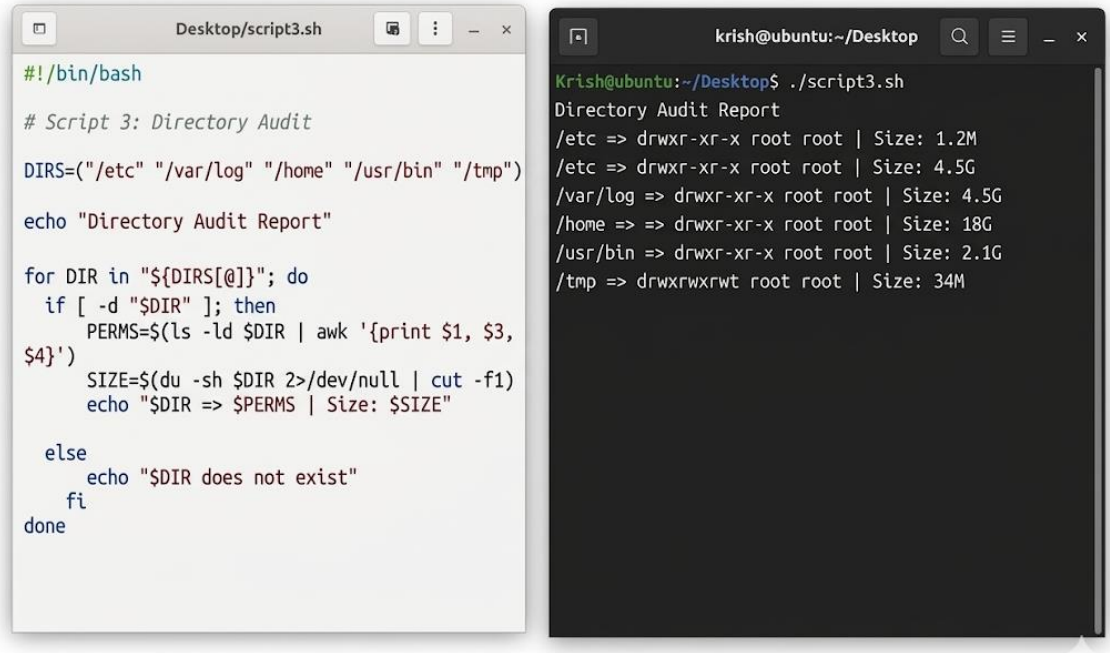


Script 3

```
#!/bin/bash
# Script 3: Directory Audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"

for DIR in "${DIRS[@]}"; do
 if [ -d "$DIR" ]; then
 PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
 SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
 echo "$DIR => $PERMS | Size: $SIZE"
 else
 echo "$DIR does not exist"
 fi
done
```

Script 3 — Disk and Permission Auditor

This script loops through a list of important system directories using a for loop. For each directory, it checks if it exists and then retrieves its size and permissions. Commands like ls -ld, du -sh, awk, and cut are used to extract specific information. This script demonstrates looping, file system inspection, and text processing.
It is useful for understanding how storage and permissions are managed in Linux.

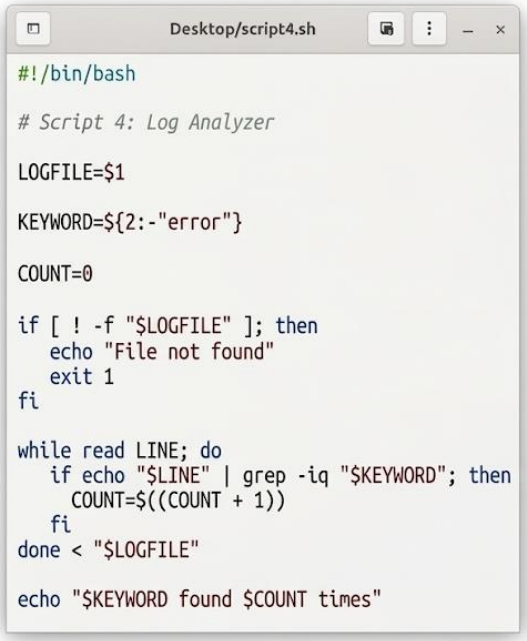


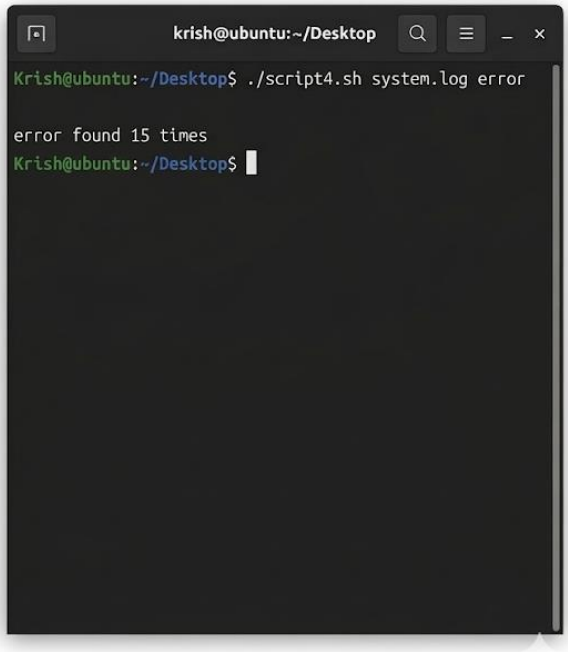
Script 4

```
#!/bin/bash
# Script 4: Log Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
 echo "File not found"
 exit 1
fi

while read LINE; do
 if echo "$LINE" | grep -iq "$KEYWORD"; then
 COUNT=$((COUNT + 1))
 fi
done < "$LOGFILE"
echo "$KEYWORD found $COUNT times"
```

Script 4 — Log File Analyzer

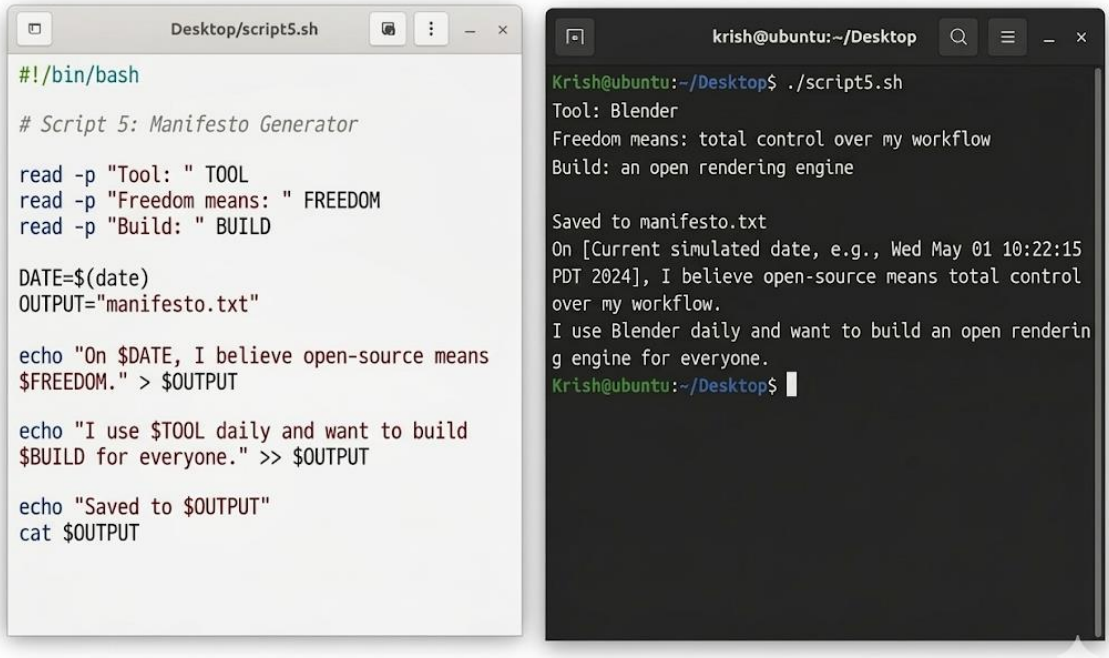
This script reads a log file line by line using a while read loop. It searches for a specific keyword (default is "error") using grep and counts how many times it appears.
It also uses command-line arguments ($1, $2) to allow user input. A counter variable is used to track occurrences.
This script demonstrates file handling, loops, conditional checks, and basic text searching, which are essential for system monitoring tasks.





Script 5

```
#!/bin/bash
# Script 5: Manifesto Generator

read -p "Tool: " TOOL
read -p "Freedom means: " FREEDOM
read -p "Build: " BUILD

DATE=$(date)
OUTPUT="manifesto.txt"

echo "On $DATE, I believe open-source means $FREEDOM." > $OUTPUT
echo "I use $TOOL daily and want to build $BUILD for everyone." >> $OUTPUT

echo "Saved to $OUTPUT"
cat $OUTPUT
```

Script 5 — Open-Source Manifesto Generator

This script interacts with the user by asking three questions using the read command. It stores the responses in variables and then combines them into a paragraph.
The output is saved to a text file using > and >>. The script also uses the date command to include the current date.
This script demonstrates user input, string handling, and file writing. It also reflects the creative side of shell scripting by generating personalized content.